

PROGRAMACIÓN 3

Programación Orientada a Objetos — Desafíos de Algoritmos

Instrucciones: Cada desafío requiere diseñar e implementar una solución en **C# .NET 10** aplicando los conceptos de Programación Orientada a Objetos. Leé el enunciado completo antes de comenzar a codificar.

■ DESAFÍO 1 — ENCAPSULAMIENTO

Pilar:
Encapsulamiento

"El Sistema Bancario Seguro"

Diseña una clase **CuentaBancaria** que proteja el saldo de modificaciones directas. El objetivo es aplicar correctamente el principio de encapsulamiento para garantizar que el estado interno del objeto solo pueda modificarse a través de métodos controlados.

El sistema debe:

- Permitir **depositar** y **retirar** dinero con validaciones (monto mayor a cero, saldo suficiente).
- Registrar un **historial de transacciones** que no pueda ser modificado desde fuera de la clase.
- Calcular **interés mensual** según el tipo de cuenta: **Ahorro = 3%**, **Corriente = 0%**.
- Impedir saldos negativos y depósitos de montos menores o iguales a cero.
- Exponer únicamente lo necesario al exterior mediante propiedades de solo lectura.
- El historial debe estar disponible como **IReadOnlyList<string>**.

■ *Pista: Investigá la diferencia entre campos privados, propiedades con getter público y el tipo IReadOnlyList<T> para proteger colecciones.*

"El Sistema de Empleados"

Una empresa tiene distintos tipos de empleados con reglas de liquidación diferentes. Construí una jerarquía de clases que modele esta situación, reutilizando el comportamiento común y especializando lo necesario en cada tipo.

El sistema debe:

- **Empleado** (clase base abstracta): nombre, legajo y sueldo base.
- **EmpleadoTiempoCompleto**: recibe un bono anual fijo de \$50.000 prorrateado mensualmente.
- **EmpleadoPartTime**: trabaja X horas/mes a \$Y por hora (sin sueldo base fijo).
- **EmpleadoContratado**: tiene fecha de vencimiento de contrato y no recibe bono.
- Cada tipo debe calcular su **sueldo total mensual** con su propia lógica.
- Mostrar la información completa de cada empleado con un método **MostrarInfo()**.
- El programa debe manejar una lista con empleados de los tres tipos y calcular el **costo total de nómina**.
- Para EmpleadoContratado, indicar cuántos días faltan para el vencimiento del contrato.

■ *Pista: Pensá qué métodos deben ser abstract y cuáles virtual. Usá : base() para reutilizar la lógica del constructor padre.*

■ DESAFÍO 3 — POLIMORFISMO

Pilar: Polimorfismo

"El Motor de Notificaciones"

Un sistema empresarial necesita enviar notificaciones por distintos canales de forma transparente. El objetivo es que el código que dispara las notificaciones no sepa ni le importe qué canal concreto se está utilizando.

El sistema debe:

- Definir la interfaz **INotificador** con el método **Enviar(string destinatario, string mensaje)**.
- Implementar tres canales concretos: **NotificadorEmail**, **NotificadorSMS** y **NotificadorPush**.
- Cada notificador debe registrar un contador de **mensajes enviados**.
- Crear la clase **GestorNotificaciones** que reciba cualquier combinación de notificadores.
- El método **NotificarATodos()** debe enviar el mensaje por todos los canales registrados.
- El Gestor no debe conocer los tipos concretos — solo trabajar con la interfaz.
- Al finalizar, mostrar un **reporte** con cuántos mensajes envió cada canal.
- Agregar un nuevo canal no debe requerir modificar la clase GestorNotificaciones.

■ Pista: Investigá el Patrón Template Method para evitar repetir el contador en cada clase. ¿Podría una clase abstracta intermedia ayudarte?

■ DESAFÍO 4 — ABSTRACCIÓN

Pilar: Abstracción

"El Sistema de Inventario con Búsqueda Genérica"

Construí un sistema de inventario capaz de gestionar productos de distintas categorías. El desafío está en modelar correctamente qué es común a todos los productos y qué es específico de cada tipo, usando clases abstractas y genéricos.

El sistema debe:

- Clase abstracta **Producto** con: código, nombre, precio y stock.
- Método abstracto **ObtenerDescripcion()** que cada subclase implementa a su manera.
- Subclase **ProductoElectronico**: agrega garantía en meses.
- Subclase **ProductoAlimento**: agrega fecha de vencimiento e indica si está vencido.
- Subclase **ProductoRopa**: agrega talle y color.
- Clase genérica **Inventario<T>** con restricción: T debe ser Producto o derivado.
- Métodos del inventario: **Agregar** (sin duplicados por código), **BuscarPorNombre** (búsqueda parcial), **FiltrarPorStockMinimo** y **calcular el valor total** del inventario.
- Demostrar el uso con los tres tipos de producto en la misma colección.

■ Pista: La restricción where $T : \text{Producto}$ garantiza que podés acceder a todas las propiedades de Producto dentro de la clase genérica.

■ DESAFÍO 5 — ALGORITMOS DE ORDENACIÓN

Pilar: Algoritmos de Ordenación

"El Sistema de Clasificación del Torneo"

Estás construyendo el sistema de clasificación para un torneo de programación. El desafío consiste en implementar algoritmos de ordenación desde cero y compararlos bajo las mismas condiciones, aplicando el patrón Strategy.

El sistema debe:

- Modelar un **Competidor** con: nombre, puntaje, tiempo de resolución (segundos) y penalizaciones.
- Implementar tres algoritmos desde cero — **sin usar .Sort() ni LINQ OrderBy**: BubbleSort, SelectionSort e InsertionSort.
- Cada algoritmo debe ser una clase que implemente la interfaz **IAlgoritmoOrdenacion** y registre el **total de comparaciones realizadas**.
- La clase **Clasificador** debe poder usar cualquier algoritmo de forma intercambiable y permitir cambiar el algoritmo en tiempo de ejecución.
- **Criterio de ordenación**: primero por puntaje descendente; en empate, por tiempo ascendente; en segundo empate, por penalizaciones ascendentes.
- Mostrar el **podio top 3**, el algoritmo usado y la cantidad de comparaciones.
- Ejecutar los tres algoritmos sobre la misma lista desordenada y comparar sus comparaciones.

■ *Pista: Encapsulá el criterio de comparación dentro de la clase Competidor con un método CompararCon(). Así los algoritmos no necesitan conocer las reglas de negocio.*

■ *Observación: ¿Cuántas comparaciones hace cada algoritmo con la lista ya ordenada vs desordenada?*

■ DESAFÍO 6 — BACKTRACKING + OOP

Pilar: Backtracking
+ OOP

"El Caballo Explorador"

Tenés un tablero numérico de 5×5. Un caballo de ajedrez parte de una posición inicial y genera series numéricas visitando celdas según las reglas del movimiento en L (2+1 o 1+2 en cualquier dirección — 8 movimientos posibles). El objetivo es explorar todos los caminos posibles usando backtracking recursivo.

El sistema debe:

- Clase abstracta **Tablero** con dos implementaciones: **TableroSecuencial** (valores 1–25) y **TableroPersonalizado** (valores definidos por el usuario).
- Clase **Caballo** que encapsule su posición y calcule sus movimientos válidos según los 8 saltos posibles del caballo de ajedrez.
- Interfaz **IFiltroDeSerie** con al menos tres filtros concretos: **FiltroSinRepetidos**, **FiltroProgresionAritmetica** y **FiltroSumaMayorA**.
- Clase **GeneradorSeries** que use **backtracking recursivo** para explorar todos los caminos posibles de longitud N desde una posición dada.
- El algoritmo de backtracking debe seguir el esquema: **Elegir** → **Explorar** → **Deshacer** en cada nivel de recursión.
- Registrar y mostrar la cantidad total de **caminos explorados** y **series generadas**.
- Aplicar cada filtro sobre las series encontradas y mostrar los resultados.
- Mostrar la serie con **mayor suma** y la de **menor suma**.
- Probar el sistema con al menos dos tableros diferentes (secuencial y personalizado).

■ *Pista:* El caso base del backtracking es cuando `movimientosRestantes == 0`. Al guardar una serie, hacé una copia — si guardás la referencia, todos los resultados apuntarán a la misma lista modificada.

■ *Recordatorio:* Los 8 saltos del caballo son combinaciones de $(\pm 1, \pm 2)$ y $(\pm 2, \pm 1)$. Descartá los que caigan fuera del tablero.

Resumen de conceptos evaluados

#	Desafío	Pilar OOP	Concepto adicional
1	Sistema Bancario Seguro	Encapsulamiento	ICollection, propiedades
2	Sistema de Empleados	Herencia	abstract, virtual, override
3	Motor de Notificaciones	Polimorfismo	Interfaces, Template Method
4	Inventario con Búsqueda Genérica	Abstracción	Genéricos con restricción (where)
5	Clasificación del Torneo	Algoritmos de Orden	Strategy Pattern, $O(n^2)$
6	El Caballo Explorador	Integrador	Backtracking recursivo, filtros